AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)

Dept. of Computer Science
Faculty of Science and Technology

CSC2210: OBJECT ORIENTED PROGRAMMING 2

Summer 2025-2026

Section: [*R*]

Group No: 00

Project Report On

Project Name ***[Hospital Management System]***

Supervised By

Dr. Md. Iftekharul Mubin

Submitted By:

Name	ID
1. S M Taj Ahmed	**24-59219-3**
2. MD.Shaiyan Bean Omar	**24-60126-3**
3. Redwan Mohd Mukles	**24-60102-3**

Obtained Marks for CO2 and CO3 (Description given in the following page)

Assessment Criteria	Not Attended/ Incorrect (0)	Inadequate (1-2)	Average (3)	Good (4)	Excellent (5)

Evaluation Criteria (CO2)	Total =	Evaluation Criteria (CO3)	Total =
Requirement fulfillment		Organization of the application	
Validation		Representation and Integration of Database	
Verification		Graphical User Interface	

CO2: Display and verify the mean of a real-life Project using the concepts of C# Graphical User Interface based environment with database integration to depict a desktop-based application.					
Assessment Criteria	**Not Attended/ Incorrect (0)**	**Inadequate (1-2)**	**Average (3)**	**Good (4)**	**Excellent (5)**
Evaluation Criteria	**Evaluation Definition**				
Requirement fulfillment	Fails to demonstrate any understanding of real-life scenario-based project development or functional requirement identification. There is no attempt to depict a project or identify functional requirements accurately.	Demonstrates limited understanding of real-life scenario-based project development and functional requirement identification. The project depicted lacks coherence or relevance to real-life scenarios, and functional requirements are inaccurately identified or insufficiently described.	Presents a basic depiction of a real-life scenario-based project and identifies some functional requirements. However, the project lacks depth or complexity, and some functional requirements may be vaguely defined or missing key details.	Effectively demonstrates a realistic scenario-based project and accurately identifies most functional requirements. The project is well-developed with appropriate complexity, and functional requirements are clearly articulated with relevant details.	Exhibits an exceptional understanding of real-life scenario-based project development and accurately identifies all functional requirements. The project is meticulously developed with thorough attention to detail, reflecting a comprehensive understanding of Object-Oriented Programming project development activities.
Validation	Fails to demonstrate any understanding or implementation of validation forms in their system. There is no attempt to deal with data validation, and validation requirements are completely ignored or incorrectly applied.	Demonstrates limited understanding of validation forms and data validation techniques. While some attempt may be made to implement validation, it is incomplete or poorly executed, leading to inadequate handling of data validation.	Shows a basic understanding of validation forms and data validation techniques. They attempt to implement validation, but some aspects may be missing or incorrectly implemented, resulting in partial or inconsistent handling of data validation.	Effectively demonstrates the use of validation forms and implements data validation techniques. Validation is mostly accurate and comprehensive, ensuring the proper handling of data input and verification in the system.	Exhibits an exceptional understanding and implementation of validation forms and data validation techniques. Validation is meticulously implemented with thorough attention to detail, ensuring robust data validation procedures and contributing to the overall reliability and integrity of the system.
Verification	Fails to demonstrate any attempt to verify the system data or functional requirements. There is no evidence of	Demonstrates limited understanding of verification processes and data flow in the system. Verification	Shows a basic understanding of verification processes and attempts to verify system data. However, verification	Identifies and verifies system data, ensuring proper functional requirements are met. Verification efforts are mostly accurate and	Exhibits an exceptional understanding of verification processes and meticulously verifies system data. Verification

	understanding or implementation of verification processes, and data flow is not considered.	attempts are incomplete or inaccurate, and there is insufficient consideration given to ensuring data integrity and functionality.	efforts may be inconsistent or lack thoroughness, and there may be gaps in ensuring proper functional requirements and data flow.	thorough, with attention to ensuring data integrity and appropriate data flow within the system.	efforts are comprehensive and precise, with a keen focus on ensuring all functional requirements are met and maintaining proper data flow throughout the system.

CO3: Prepare and Explain a real life desktop based application synthesizing several component of C# along with development tools to adhere the given requirements.

Assessment Criteria	Not Attended/ Incorrect (0)	Inadequate (1-2)	Average (3)	Good (4)	Excellent (5)
Evaluation Criteria	**Evaluation Definition**				
Organization of the application	Fails to identify any suitable real time application or requirements for project development activities related to OOP.	Limited understanding about the project scopes and scenarios or identification of functional requirements.	Lacks depth or relevance to OOP project development activities and may contain inaccuracies. Real-life scenarios are mentioned, but the discussion lacks depth or clarity.	Consider and integrate the idea of several core aspects of the project along with relevance to real-life scenarios. Demonstrating a solid understanding of the application presentation.	Generalize and exhibits an exceptional understanding of project preparation according to a to real-life scenarios. Also contains proper and insightful identification of the system which is comprehensive and precise.
Representation and Integration of Database	Fails to identify and present any understanding or implementation of database. Also failed to integrate the data with the project itself.	Limited understanding of the database concepts or their proper way of using in a real time project. While some attempt may be made to implement but it is incomplete or poorly executed, leading to inadequate design.	Lacks depth or relevance to database integration with the application. Shows a basic understanding but some aspects may be missing or incorrectly implemented, resulting in partial or inconsistency. May lack proper normalization.	Integrate the database with the forms properly and implements it with proper validation which is mostly accurate and comprehensive, ensuring the proper handling of data input and verification along with general normalization.	Exhibits an exceptional understanding and implementation of database ensuring attention to detail, and robust data manipulation procedures and contributing to the overall clarity.
Graphical User Interface	Fails to present or prepare GUI based application interfaces. There is no evidence of creating or integrating such things according to their usefulness.	Limited understanding of graphical user interfaces. Lack of design knowledge. Very poor attempt to make such things which are currently obsolete	Shows a basic understanding of creating user interfaces. Most of them are interconnected but maybe some of them lack it. However, most of it can be	Effectively identifies and meet the consider the simplicity. Design related works are mostly accurate and taken proper attention to ensuring a user-	Exhibits an exceptional work design following a high standard of simple and elegant work. Several controls and mechanism has been organized in a

		or can't be identified as coherent.	described as user friendly.	friendly coherent system.	preferred way according to the coherent usage .

Table of Contents: Page no.

Case Study:

In a comprehensive Hospital Management System, patients can register with the hospital and access different healthcare services according to their needs. Patients can book appointments with doctors, view their medical history, prescriptions, laboratory reports, and bills.

Each patient is uniquely identified by a Patient ID and has information such as name, email, phone number, address, date of birth, gender, and blood group.

Every doctor in the hospital is identified by a Doctor ID and has details such as name, specialization, department, contact number, and availability. Doctors can examine patients, record diagnoses, prescribe medicines, and request laboratory tests.

The system also contains nurses, laboratory technicians, pharmacists, and accountants, each responsible for their respective hospital activities. Nurses can monitor admitted patients, laboratory technicians can manage test results, pharmacists can manage medicines, and accountants can handle billing and payments.

Patients who require hospitalization can be assigned a room and bed. The system keeps track of available and occupied beds and records the patient's admission and discharge information.

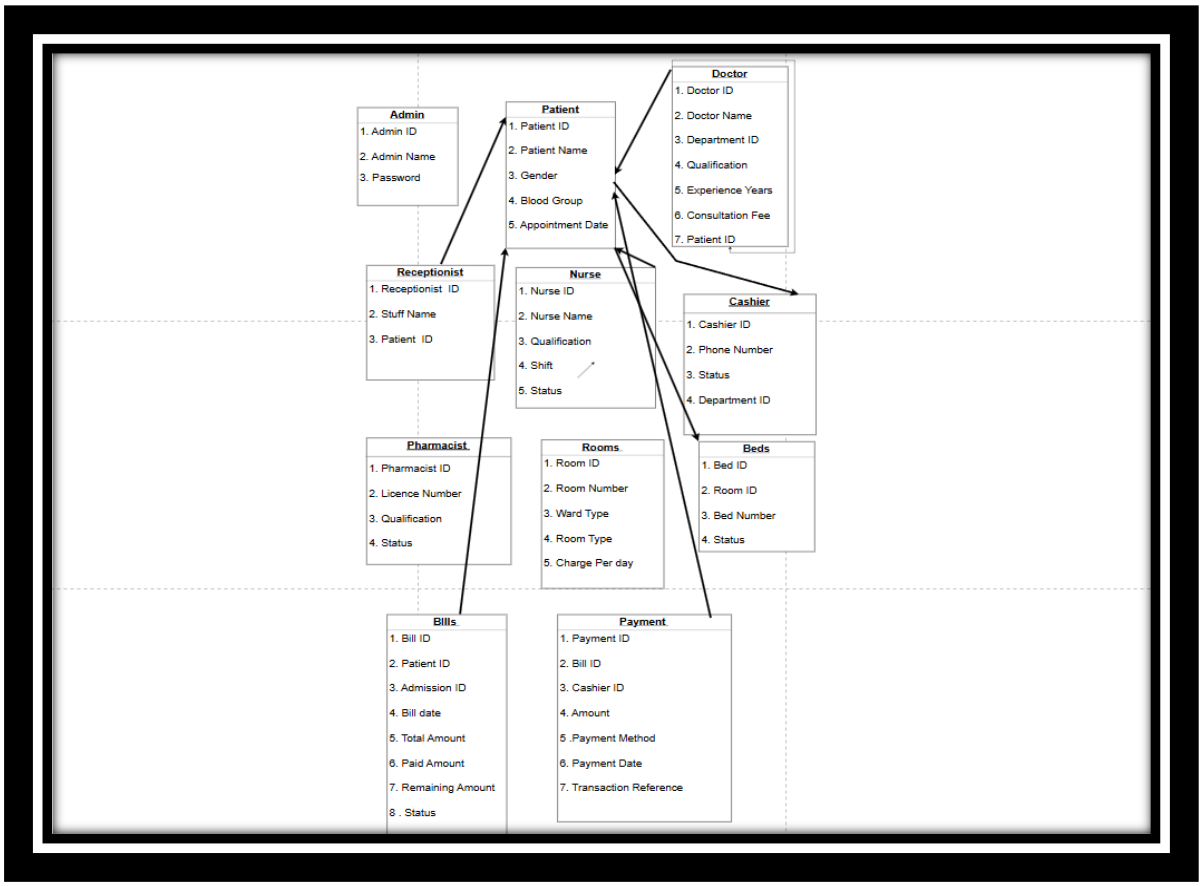
The system operates under the supervision of an administrator, who manages users, doctors, staff, departments, rooms, medicines, and other hospital information while maintaining the security of the system.

Functionality:

1.After Login, users can access the system according to their role such as Admin, Doctor, Nurse, Receptionist, Pharmacist, Lab Technician, Accountant, or Patient.
2.Patients can register and manage their account, including viewing and updating their personal information.
3.Patients can book, cancel, or reschedule appointments with available doctors.
4.Doctors can view patients, medical history, add diagnosis, prescribe medicines, and request laboratory tests.
5.Laboratory technicians can receive test requests and add test results which can be viewed by the doctor and patient.
6.Pharmacists can manage medicines and stock and dispense medicines according to the doctor's prescription.
7.Nurses can manage admitted patients by recording vital signs, medication administration, and nursing notes.
8.The system can manage hospital admission, rooms, and beds, including assigning available beds and releasing them after discharge.
9.Accountants can generate bills and manage payments for consultation, tests, medicines, rooms, and other hospital services.
10.Patients can view their medical records, prescriptions, laboratory results, appointments, bills, and payment history.
11.Admin can view and manage all users, doctors, staff, patients, departments, rooms, medicines, appointments, and hospital records.

12.Admin can deactivate or remove users, doctors, staff, or other records when necessary or for unauthorized/inappropriate activities

DATABASE SCHEMA :



CREATE DATABASE Hospital_Management_System;
USE Hospital_Management_System;

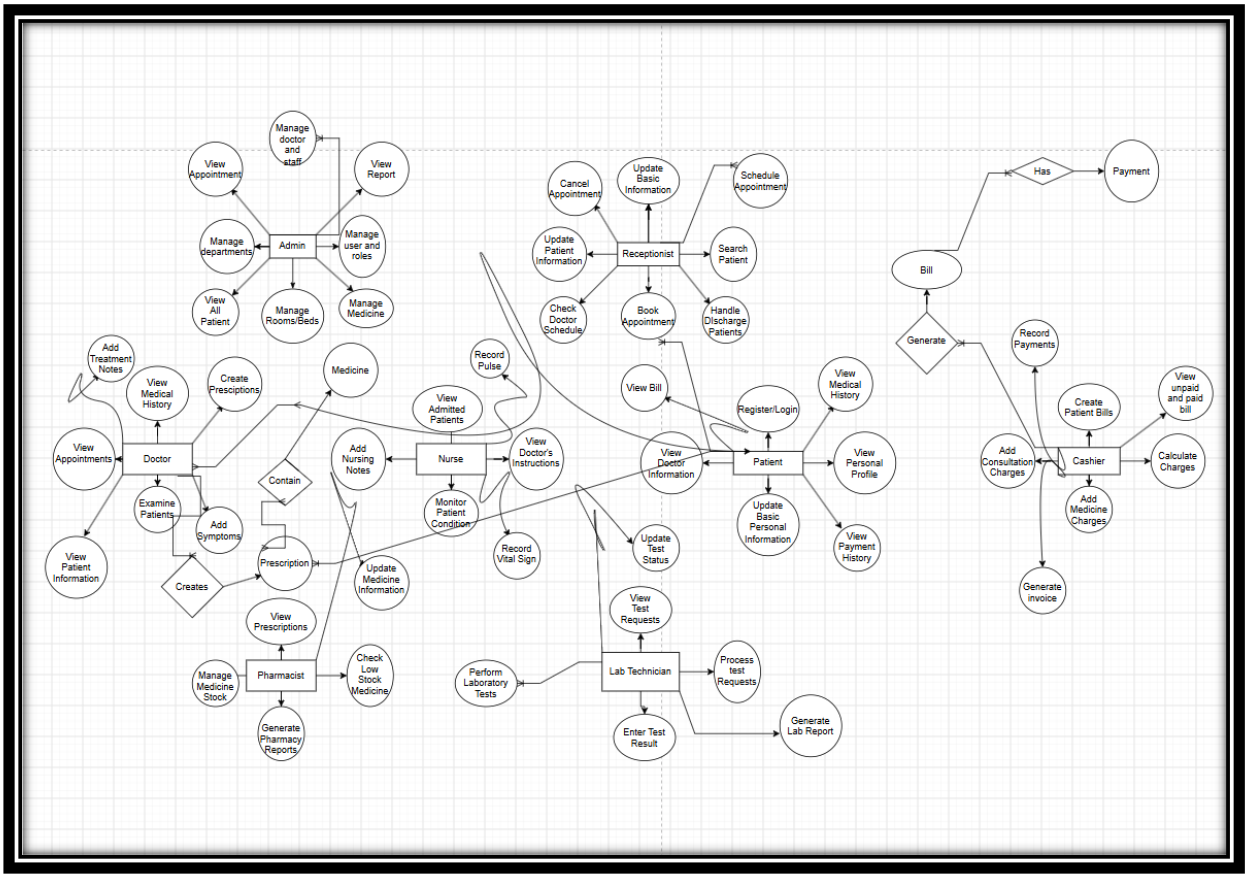
CREATE TABLE System_User (
    User_ID INT AUTO_INCREMENT PRIMARY KEY,
    Username VARCHAR(64) UNIQUE NOT NULL,
    Password VARCHAR(64) NOT NULL,
    Role ENUM('Admin', 'Doctor', 'Nurse', 'Lab Technician', 'Pharmacist', 'Accountant', 'Receptionist', 'Patient') NOT NULL,
    Created_At TIMESTAMP DEFAULT CURRENT_TIMESTAMP

```
);
CREATE TABLE Admin (
   Admin_ID NUMBER PRIMARY KEY,
   Admin_Name VARCHAR2(100) NOT NULL,
   Password VARCHAR2(100) NOT NULL
);

CREATE TABLE Patient (
   Patient_ID NUMBER PRIMARY KEY,
   Patient_Name VARCHAR2(100) NOT NULL,
   Gender VARCHAR2(20),
   Blood_Group VARCHAR2(10),
   Appointment_Date DATE
);

CREATE TABLE Doctor (
   Doctor_ID NUMBER PRIMARY KEY,
   Doctor_Name VARCHAR2(100) NOT NULL,
   Department_ID NUMBER,
   Qualification VARCHAR2(100),
   Experience_Years NUMBER,
   Consultation_Fee NUMBER(10,2)
);

CREATE TABLE Doctor_Patient (
   Doctor_ID NUMBER,
   Patient_ID NUMBER,

   PRIMARY KEY (Doctor_ID, Patient_ID),

   CONSTRAINT FK_DP_DOCTOR
      FOREIGN KEY (Doctor_ID)
      REFERENCES Doctor(Doctor_ID),

   CONSTRAINT FK_DP_PATIENT
      FOREIGN KEY (Patient_ID)
      REFERENCES Patient(Patient_ID)
);

CREATE TABLE Receptionist (
   Receptionist_ID NUMBER PRIMARY KEY,
   Staff_Name VARCHAR2(100) NOT NULL,
   Patient_ID NUMBER,

   CONSTRAINT FK_RECEPTIONIST_PATIENT
      FOREIGN KEY (Patient_ID)
      REFERENCES Patient(Patient_ID)
);

CREATE TABLE Nurse (
```

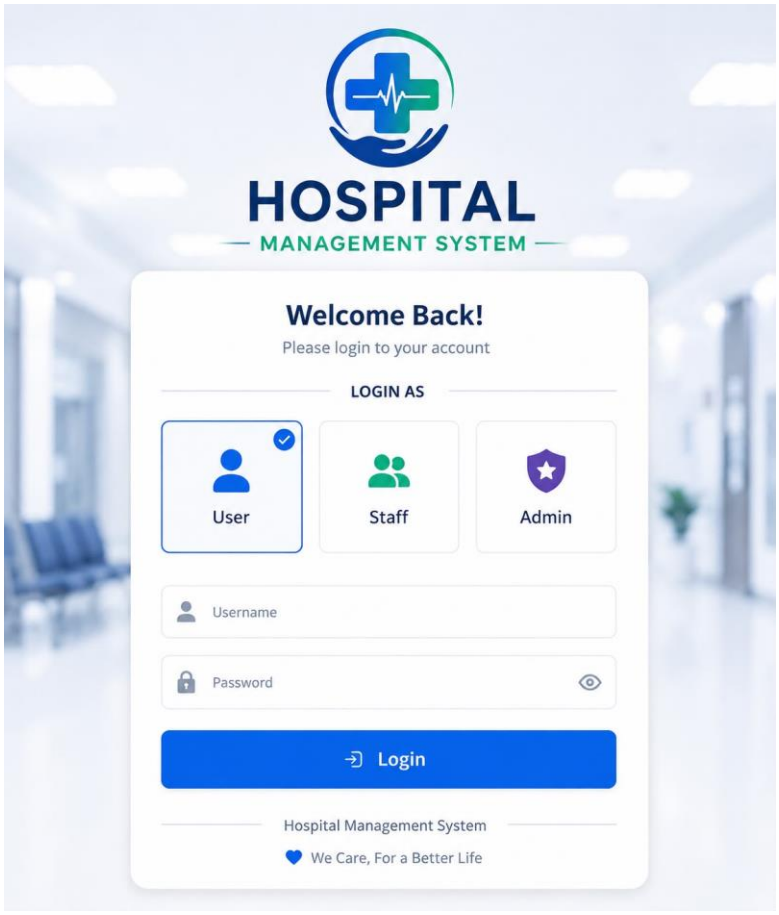
```
    Nurse_ID NUMBER PRIMARY KEY,
    Nurse_Name VARCHAR2(100) NOT NULL,
    Qualification VARCHAR2(100),
    Shift VARCHAR2(50),
    Status VARCHAR2(30)
);

CREATE TABLE Cashier (
    Cashier_ID NUMBER PRIMARY KEY,
    Phone_Number VARCHAR2(20),
    Status VARCHAR2(30),
    Department_ID NUMBER
);


CREATE TABLE Pharmacist (
    Pharmacist_ID NUMBER PRIMARY KEY,
    Licence_Number VARCHAR2(50) UNIQUE,
    Qualification VARCHAR2(100),
    Status VARCHAR2(30)
);


CREATE TABLE Rooms (
    Room_ID NUMBER PRIMARY KEY,
    Room_Number VARCHAR2(20) UNIQUE,
    Ward_Type VARCHAR2(50),
    Room_Type VARCHAR2(50),
    Charge_Per_Day NUMBER(10,2)
);

CREATE TABLE Beds (
    Bed_ID NUMBER PRIMARY KEY,
    Room_ID NUMBER NOT NULL,
    Bed_Number VARCHAR2(20),
    Status VARCHAR2(30),

    CONSTRAINT FK_BED_ROOM
        FOREIGN KEY (Room_ID)
        REFERENCES Rooms(Room_ID)
);


CREATE TABLE Bills (
    Bill_ID NUMBER PRIMARY KEY,
    Patient_ID NUMBER NOT NULL,
    Admission_ID NUMBER,
    Bill_Date DATE,
    Total_Amount NUMBER(10,2),
    Paid_Amount NUMBER(10,2),
```

```
    Remaining_Amount NUMBER(10,2),
    Status VARCHAR2(30),

    CONSTRAINT FK_BILL_PATIENT
        FOREIGN KEY (Patient_ID)
        REFERENCES Patient(Patient_ID)
);


CREATE TABLE Payment (
    Payment_ID NUMBER PRIMARY KEY,
    Bill_ID NUMBER NOT NULL,
    Cashier_ID NUMBER NOT NULL,
    Amount NUMBER(10,2),
    Payment_Method VARCHAR2(30),
    Payment_Date DATE,
    Transaction_Reference VARCHAR2(100),

    CONSTRAINT FK_PAYMENT_BILL
        FOREIGN KEY (Bill_ID)
        REFERENCES Bills(Bill_ID),

    CONSTRAINT FK_PAYMENT_CASHIER
        FOREIGN KEY (Cashier_ID)
        REFERENCES Cashier(Cashier_ID)
);

SELECT * FROM System_User;
SELECT * FROM Doctor;
SELECT * FROM Patient;
SELECT * FROM Room;
SELECT * FROM Bed;
```

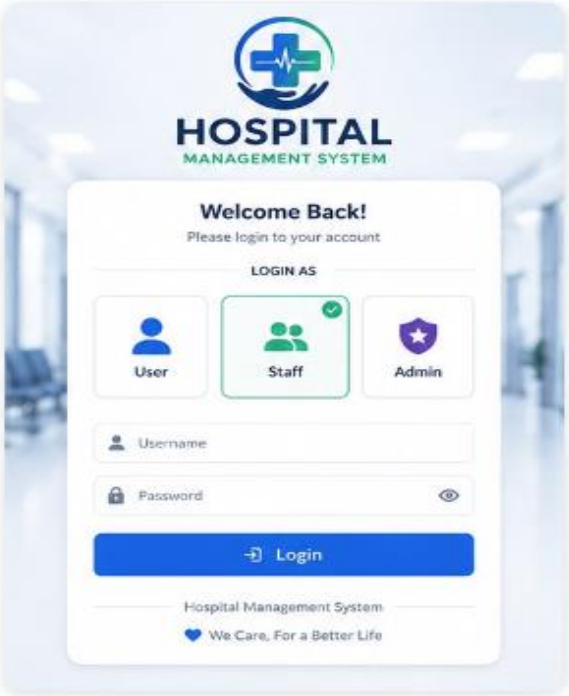
ER Diagram:


Manage doctor and staff
View Appointment
View Report
Manage departments
Admin
Manage user and roles
View All Patient
Manage Rooms/Beds
Manage Medicine
Cancel Appointment
Update Basic Information
Schedule Appointment
Update Patient Information
Receptionist
Search Patient
Check Doctor Schedule
Book Appointment
Handle Discharge Patients
Has
Payment
Bill
Generate
Record Payments
Add Treatment Notes
View Medical History
Create Presciptions
Medicine
Record Pulse
View Admitted Patients
View Bill
Register/Login
View Medical History
View Appointments
Doctor
Add Nursing Notes
Nurse
View Doctor's Instructions
View Doctor Information
Patient
View Personal Profile
Create Patient Bills
View unpaid and paid bill
Add Consultation Charges
Cashier
Calculate Charges
Examine Patients
Add Symptoms
Contain
Monitor Patient Condition
Update Basic Personal Information
Add Medicine Charges
View Patient Information
Creates
Prescription
Record Vital Sign
Update Test Status
View Payment History
Update Medicine Information
Generate invoice
View Test Requests
View Prescriptions
Manage Medicine Stock
Pharmacist
Check Low Stock Medicine
Perform Laboratory Tests
Lab Technician
Process test Requests
Generate Pharmacy Reports
Enter Test Result
Generate Lab Report

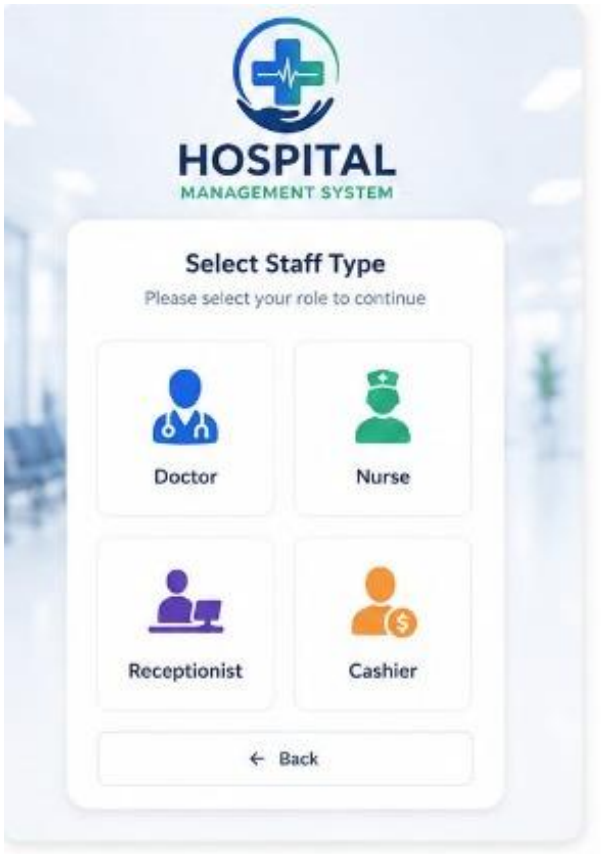
Login Page:



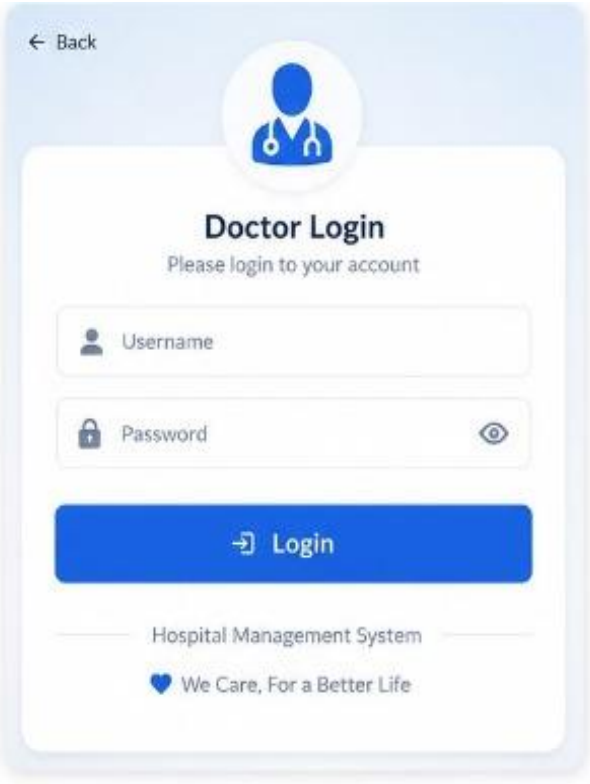
If click on staff option then this showing :



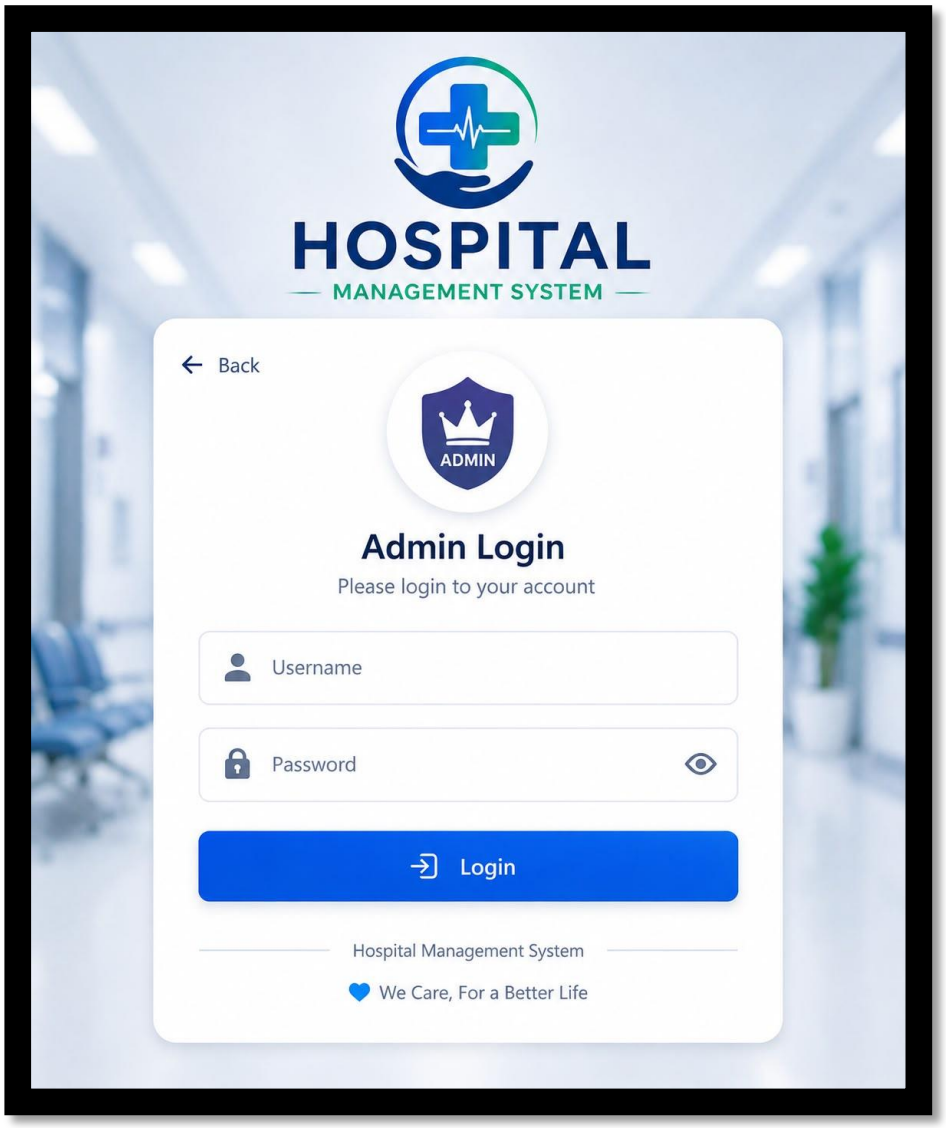
Then this interface coming :



Now if I select Doctor then this interface coming:



Now if we select admin option then we see:



Work Distribution Table:

Name	ID	Contribution
1.Md.Shaiyan Bean Omar	24-60126-3	Form 1 to 4
2.Redwan Mohd Mukles	24-60102-3	Form 5 to 8
3.S M Taj Ahmed	24-59219-3	Form 9

Conclusion:

The Online Hospital Management System successfully establishes a secure, structured, and user-centric platform for streamlining core medical operations. By integrating a relational database architecture with a C# Windows Forms frontend, the system connects distinct user roles—Admins, Doctors, Cashiers, and Patients—into a single digital ecosystem.
Through features such as role-based login validation, automated daily appointment tracking, electronic patient history retrieval, and dynamic medical billing, the application eliminates manual recordkeeping errors and operational delays. Safe SQL practices and object-oriented design patterns ensure that sensitive healthcare data remains structured and scalable. Ultimately, the system bridges the gap between clinical care and administrative management, delivering an efficient, transparent experience for hospital staff and patients alike.

Feature Work:

This platform establishes a solid foundation for digitized hospital workflows through its role-based architecture for Admins, Doctors, Patients, and Cashiers. Future iterations can easily expand upon this structure to incorporate advanced features such as telemedicine, online payment gateway integrations, and AI-driven diagnostic assistance.